

- For **Challenge 2**, we propose *heuristic re-ranking* to eliminate unnecessary I/Os and computations during re-ranking.
- For **Challenge 3**, we propose *redundancy-aware I/O deduplication* based on the optimized storage layout to further enhance I/O efficiency.
- We evaluate FusionANNS using a general purpose server equipped with an entry-level GPU. Experimental results show that FusionANNS improves QPS by up to $13.1\times$ and $4.9\times$, and enhances cost efficiency by up to $8.8\times$ and $6.8\times$, compared with the state-of-the-art SSD-based system–SPANN and GPU-accelerated in-memory system–RUMMY, respectively, while guaranteeing low latency and high accuracy.

2 Background and Motivation

In this section, we first introduce two kinds of ANNS indexing techniques and product quantization (PQ) for vectors. Then, we present our main idea and analyze its key challenges, which motivate the design of FusionANNS.

2.1 Indexing Techniques for ANNS

Most ANNS algorithms exploit a distance metric such as Euclidean distance to find the top- k nearest neighbors for a given query vector. For high-dimensional large-scale datasets, it is computationally costly due to the curse of dimensionality [27]. To address this issue, most ANNS algorithms [16, 17, 28–32] exploit indexing techniques to prune data regions that are unlikely to contain the nearest neighbors. These indices can significantly improve the query performance by shrinking the search space, but significantly increases memory consumption, especially for large datasets. Among various indexing techniques, IVF [15, 16, 31, 32] and graph-based [17, 33] indices are widely used due to their high efficiency.

The **graph-based index** often organizes vectors in a proximity graph structure, in which vertices and edges represent vectors and distances between two vertices, respectively. Upon a query, the ANNS engine traverses the graph from a given vertex to find the top- k nearest neighbors. DiskANN [22] is a typical graph-based ANNS solution. It uses SSDs to store graph indices of billion-scale datasets while keeping some frequently-accessed vertices in main memory. Although DiskANN is a memory-efficient ANNS solution, it experiences high latency for queries due to rather long iteration paths for large-scale datasets.

The **IVF index** is a popular indexing technique for large-scale datasets stored on SSDs. To create the IVF index, a dataset is often partitioned into many posting lists using a clustering algorithm [34], and each posting list is represented by its centroid. Recent studies [9, 35] have demonstrated that the IVF index [15] is more efficient than the state-of-the-art graph-based index [22] for billion-scale datasets. SPANN [15]

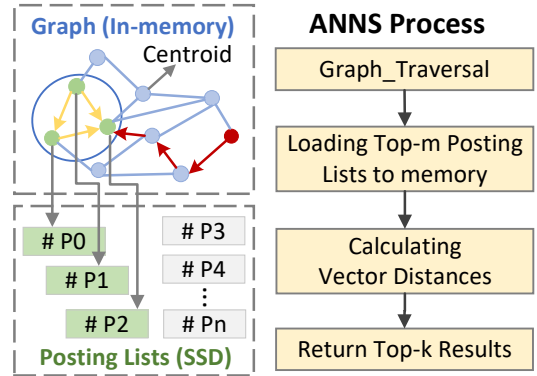


Figure 2: The hierarchical indexing technique in SPANN

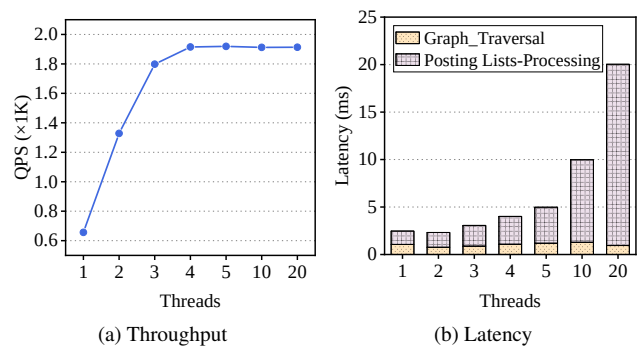


Figure 3: The throughput and latency of SPANN

is a state-of-the-art billion-scale ANNS system using the IVF index. Unlike conventional IVF indices, SPANN builds an advanced IVF index by replicating boundary vectors of clusters into adjacent posting lists. This replication mechanism significantly expands the size of vector indices by $8\times$, but improves the query accuracy and efficiency. As shown in Figure 2, SPANN stores all posting lists on SSDs while maintaining the centroids of these posting lists in memory using a graph index. Upon a query, SPANN traverses the in-memory graph to identify top- m nearest posting lists and loads them to host main memory. Then, it finds the top- k ($k < m$) nearest neighbors within these m posting lists via distance calculations.

Although SPANN achieves low latency comparable to in-memory ANNS approaches, we find that its throughput for concurrent queries is quite limited. As shown in Figure 3a, SPANN achieves the peak QPS using only four CPU threads, and its throughput can not scale with more threads. We count the query latency in two stages: (i) graph traversal in memory, and (ii) processing posting lists from SSD. Figure 3b shows that the query latency increases almost linearly with the number of threads. However, the latency of graph traversal almost remains stable, whereas the latency of processing posting lists increases significantly with the number of threads. The reason is that multiple queries concurrently read many and large-size posting lists from SSDs, resulting in severe I/O contention and high latency.

2.2 Product Quantization

To reduce the size of vector indices and computational costs for large-scale datasets, *product quantization* (PQ) [32] has been explored recently for compressing high-dimensional vectors. Assume a dataset containing N vectors is compressed with PQ, these vectors are first divided evenly into M sub-spaces, and each contains N sub-vectors. Then, these sub-vectors are clustered to generate a codebook, which contains a set of centroids of all clusters. The codebook allows each sub-vector to be approximated by its nearest centroid. The number of clusters per sub-space is typically set to 256, allowing each cluster ID to be represented by one byte. Once all codebooks are generated, each vector can be compressed into an M -byte PQ code. Upon a query, a distance lookup table is first generated, including all distances between a sub-query-vector and centroids per sub-space. Then, the approximate distance between the query vector q and a compressed vector v can be formulated as:

$$\widehat{dist}(q, v) = \sum_{i=1}^M dist(q_i, c_i(v_i)) \quad (1)$$

where M denotes the total number of sub-spaces, q_i denotes the i -th sub-query-vector, and $c_i(v_i)$ denotes the centroid of the i -th sub-space of the compressed vector. Thus, the distance between q_i and $c_i(v_i)$ can be easily retrieved by looking up the distance table using the PQ code as the address. Finally, the distance between q and v can be summed up with all $dist(q_i, c_i(v_i))$.

Essentially, PQ converts a distance calculation between vectors into multiple memory access operations, and thus poses a significant challenge for traditional CPU-based computing architectures due to the relatively low bandwidth of DRAM and limited parallelism. Therefore, the PQ is usually accelerated by GPUs [21, 32] because it can fully utilize their massive GPU kernels and high bandwidth memory to improve the query performance.

2.3 Main Idea and Challenges

To circumvent the challenges of substantial computing and memory resource requirements posed by billion-scale datasets, our goal is to design a high-throughput, low-latency, cost-efficient, and high-accuracy ANNS system using SSDs and an entry-level GPU. However, a significant challenge for designing a GPU-accelerated ANNS system is that the limited capacity of GPU's HBM causes extensive data swapping between GPU and CPUs, significantly degrading the ANNS performance for large-scale datasets.

A Straightforward Solution using PQ and HI. Fortunately, the PQ technique can significantly reduce the memory footprint of vectors, thereby alleviating the performance bottleneck associated with data transmission between CPUs and GPUs. As a result, PQ has the potential to fully harness the capabilities of GPUs to accelerate distance calculations involved in ANNS [20]. Here, we first discuss a straightforward

GPU-accelerated ANNS solution using PQ and *hierarchical indexing* (HI) techniques. Except that all vectors are compressed using PQ, this straightforward solution uses the same hierarchical indices as SPANN. Upon a query, the ANNS engine first traverses the navigation graph to identify top- m nearest posting lists, and loads these compressed posting lists to the GPU's HBM for distance calculations. Then, GPU finds the top- n candidate vectors by calculating the distance between the query vector and each compressed vector in the top- m nearest posting lists. Since PQ has a negative impact on the query accuracy, these intermediate results obtained by the GPU should be re-ranked to improve the query accuracy. During re-ranking, the raw data of the top- n candidate vectors should be compared with the query vector to find the final top- k nearest neighbours.

Observations. Disappointingly, we find that the above solution does not achieve expected high performance. To better understand the root causes, we conduct four different experiments to evaluate three combinations of HI, PQ, and GPU acceleration techniques. In all experiments, different ANNS systems have to meet the same level of query accuracy. As shown in Figure 4a, neither the PQ nor the GPU acceleration can reduce the end-to-end query latency compared with the HI proposed by SPANN. Although "HI+GPU" can significantly reduce the latency of distance calculations, the overhead of transferring posting lists between CPUs and the GPU (i.e., *CudaMemcpy*) offsets the benefits of the GPU acceleration. For "HI+PQ", it still uses CPUs to process PQ-based posting lists. Since vectors are compressed using PQ, the I/O latency due to loading PQ-based posting lists from SSDs to main memory is reduced. However, the CPU faces a new challenge in calculating distances between the query vector and compressed vectors due to intensive memory accesses, resulting in a significant increase of the end-to-end query latency. For "HI+PQ+GPU", the latency of distance calculations is reduced to an extremely low level. However, the *CudaMemcpy* and the additional re-ranking process incur substantial overheads, offsetting the benefits of using GPU. Moreover, none of these combinations achieve higher throughput than the original SPANN using HI solely, as shown in Figure 4b. Particularly, a direct adoption of PQ to SPANN even significantly reduces its QPS by 65%.

Root Causes. To reveal the root cause of such performance degradation, we measure the I/O numbers and the data volume transferred across SSDs, main memory, and GPU's HBM required by each ANNS query on average. As shown in Figure 4c, although the PQ technique significantly reduces the I/O size of posting lists from 12~48 KB to a page granularity (4 KB), it increases the number of I/Os by 70% due to the re-ranking process. As a result, the I/O performance bottleneck shifts from the SSD's bandwidth to its *input/output operations per second* (IOPS). Moreover, a large volume of posting lists are transferred between CPUs and GPU, thereby offsetting the benefit of GPU acceleration, as shown in Figure 4d.

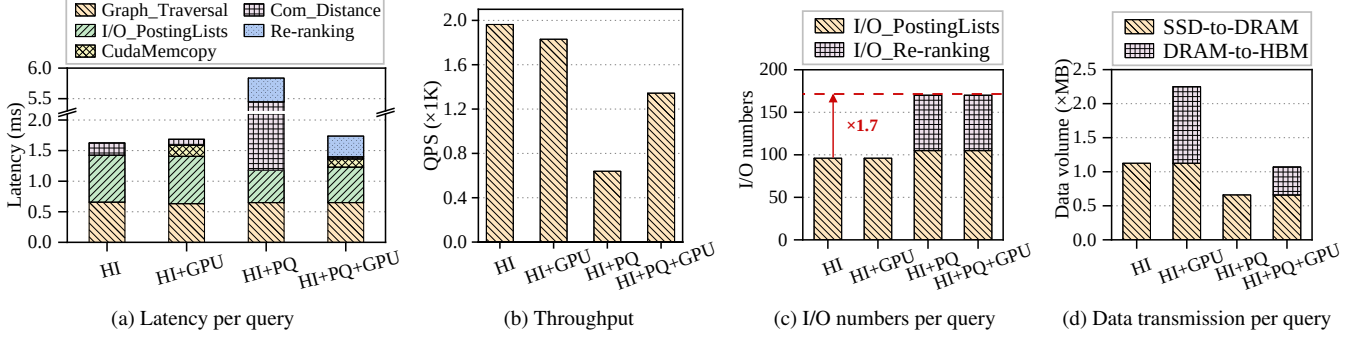


Figure 4: Three combinations of *hierarchical indexing* (HI), *product quantization* (PQ), and GPU acceleration

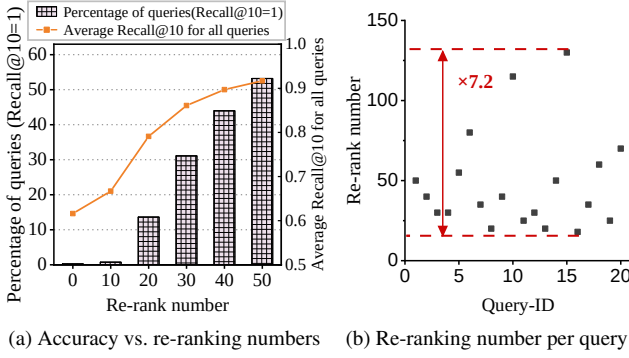


Figure 5: Differential characterization between queries

Challenge 1: Overall, the combinations of HI, PQ, and GPU acceleration techniques cause even higher latency and lower throughput than SPANN that adopts HI solely. The root cause is that the GPU’s HBM still cannot accommodate all posting lists compressed by PQ, allowing extensive data transmission between GPU and CPUs to become a new performance bottleneck. **Without a sophisticated design of the data layout across different devices and a careful collaboration among these three techniques, it is impossible to fully realize the GPU’s potential for ANNS acceleration.**

Challenge 2: To achieve the same level of query accuracy, a re-ranking process is usually required to refine intermediate results generated by the GPU. The number of the top- n vectors that should be re-ranked (i.e., the re-ranking number) is usually several times larger than the final top- k nearest neighbors. To evaluate the impact of the re-ranking number on the query accuracy, we execute 10,000 queries by linearly increasing the re-ranking number. As shown in Figure 5a, when the re-ranking number is set to 40, about 42% of queries get the accurate top-10 nearest neighbors under $Recall@10 = 1.0$, while all queries achieve an accuracy level¹ of $Recall@10 = 0.9$ on average. In this case, it is only beneficial to increase the re-ranking number for straggler queries. Moreover, we find that the minimum re-ranking numbers are very distinct for different ANNS queries, as shown in Figure 5b. This signifi-

¹ $Recall@k$ refers to the proportion of relevant vectors that are retrieved within the ground-truth top k nearest neighbors.

cant variance usually causes unnecessary I/O operations and distance calculations if the number of re-ranked vectors is fixed for all queries. However, **it is challenging to determine the minimum re-ranking number for each query under a given accuracy constraint.**

Challenge 3: The re-ranking process introduces a number of I/O requests to raw vectors on SSDs. The size of a raw vector generally ranges from 128 to 384 bytes, while the smallest operating unit of modern NVMe SSDs is typically a page (4 KB). **This mismatch in granularity causes significant read amplification, resulting in extremely low I/O efficiency during re-ranking.** Fortunately, we find these vectors requiring re-ranking usually are highly similar to each other. This similarity offers an opportunity to mitigate the read amplification by reorganizing the data layout on SSDs.

3 FusionANNS Design

In this section, we present key designs of FusionANNS, i.e., multi-tiered indexing, CPU/GPU collaborative filtering, heuristic re-ranking, and redundant-aware I/O deduplication.

3.1 Multi-tiered Indexing

Figure 6 illustrates the structure of multi-tiered indices residing in host main memory, GPU’s HBM, and SSDs. We construct multi-tiered indices in an offline manner. At first, we adopt the hierarchical balanced clustering algorithm [34] to iteratively partition the dataset into several posting lists. Each posting list contains multiple vector-IDs and the corresponding vector content. We follow SPANN to configure the number of posting lists (about 10% of the total number of vectors in a datasets) and use the same replication mechanism to address the boundary concern [15]. Specifically, when a vector lies on the boundary of multiple clusters, we assign this boundary vector to a cluster according to Equation 2.

$$v \in C_i \Leftrightarrow \text{Dist}(v, C_i) \leq (1 + \epsilon) \times \text{Dist}(v, C_1) \quad (2)$$

where v represents the vector to be assigned, C_i denotes the i -th clusters. Particularly, C_1 represents the cluster that is clos-